

Résumé simple — One-to-One Skew-GRAM

One-to-One Skew-GRAM — Résumé simple du projet

1. Le problème qu'on essaie de résoudre

Imaginez deux réseaux qui décrivent les **mêmes** éléments biologiques (par exemple des gènes ou des métabolites), mais de deux façons différentes :

- **Le graphe D (orienté)** : représente un **enchaînement de cause à effet**, comme une chaîne de réactions chimiques dans une cellule (A produit B, B produit C, etc.).
- **Le graphe G (non orienté)** : représente des **associations fonctionnelles**, comme des protéines qui interagissent entre elles ou des gènes proches sur le génome.

Le but : trouver le **plus long chemin possible** qui respecte les deux réseaux en même temps :

- Ce doit être un chemin valide dans D (on suit bien les flèches, dans l'ordre).
- Tous les éléments de ce chemin doivent former un groupe "connecté" dans G (ils doivent tous se toucher, directement ou indirectement, dans le réseau fonctionnel).

C'est un problème d'**optimisation combinatoire NP-difficile** : le nombre de chemins possibles explose très vite avec la taille du réseau, donc on ne peut pas tous les essayer un par un.

Pourquoi c'est utile ? Trouver un tel chemin revient à découvrir un **module biologique conservé** : un ensemble de gènes/métabolites qui à la fois suivent une logique causale ET sont fonctionnellement liés — un signal fort en bio-informatique (études de maladies, génomique comparée, etc.).

2. Les solutions qui existaient déjà

Méthode	Résultat	Problème
ILP2 (solveur exact)	Trouve TOUJOURS la meilleure réponse possible (l'optimum)	Très lent : ~28 secondes par instance
GNN (réseau de neurones, état de l'art)	Rapide	Très mauvais : chemins de seulement ~1,2 nœuds en moyenne (quasi inutilisable)

Il manquait une méthode **rapide ET de bonne qualité**.

3. Notre méthode : One-to-One Skew-GRAM

L'idée s'inspire d'une technique bien connue en traitement du langage (**Word2Vec / Skip-Gram**, utilisée pour apprendre le sens des mots) et l'adapte aux graphes :

1. **On "lit" le graphe D comme un texte.** On génère des milliers de petites promenades aléatoires en suivant les flèches de D — un peu comme des "phrases" faites de nœuds au lieu de mots.
2. **On entraîne un modèle qui apprend la position de chaque nœud** dans un espace mathématique (embeddings), de façon à ce que des nœuds "proches" dans D aient des positions proches. On utilise en plus des "faux exemples" intelligents (négatifs structurels) pour que l'apprentissage respecte bien la logique de D.
3. **On construit le chemin, guidé par 3 informations à la fois :**
 - o Est-ce que ce prochain nœud risque de mener à une impasse dans D ? (calculé exactement, à l'avance, avec une méthode classique de programmation dynamique)
 - o Qu'est-ce que le modèle appris pense être un bon nœud suivant ?
 - o Est-ce que ce nœud reste bien connecté au reste du chemin dans G ?
4. **On vérifie et corrige à la fin :** on extrait la plus longue portion du chemin obtenu qui respecte vraiment, à 100%, la contrainte de connexité dans G.

Point clé découvert pendant le projet : une première version du décodage (guidée seulement par le modèle appris, sans l'étape 3-a) tombait très souvent dans des impasses du graphe D — elle produisait des chemins bien trop courts.

Ajouter le calcul exact “quelle est la plus longue suite possible à partir d’ici” a résolu ce problème et a fait plus que doubler la qualité des résultats.

4. Résultats (sur 38 instances de test, 100 nœuds chacune)

Méthode	Longueur moyenne trouvée	Temps moyen	Vitesse vs ILP2
GNN (état de l’art)	~1,2 nœuds	—	—
ILP2 (optimum exact)	9,97 nœuds	28 secondes	référence
Notre méthode (Skew-GRAM)	7,13 nœuds	2,7 secondes	~10× plus rapide

- On atteint ~72% de la qualité de la solution optimale (ILP2), en environ **10 fois moins de temps**.
- Sur **13 des 36 instances testées**, notre méthode trouve un chemin **aussi long ou plus long** que la solution “optimale” exacte.
- On surpasse très largement l’ancienne méthode par réseau de neurones (1,2 nœud → 7,13 nœuds).

5. Ce qu’on a livré

- Le notebook (`one_to_one_skewgram.ipynb`) contient uniquement cette méthode Skew-GRAM (les anciennes approches concurrentes ont été retirées pour ne pas prêter à confusion).
- Les résultats chiffrés sont dans `results/summary.csv` et `results/per_instance_results.csv`.
- Le README et l’article scientifique (versions anglaise et française) ont été mis à jour pour décrire fidèlement cette méthode et ces résultats.

6. Limite connue et piste d’amélioration

Sur quelques instances, les résultats restent en retrait car la méthode utilise du **tirage aléatoire** : avec une autre “graine” de hasard, les mêmes instances donnent des résultats bien meilleurs. Ce n’est donc pas un défaut de la logique de l’algorithme, mais une question de chance dans l’exploration — une piste future serait d’essayer plusieurs graines par instance et de garder le meilleur résultat.